Universidad Nacional de Las Ciencias
Ingeniería en Ciberseguridad
Programación
Alumnos:
Nelson Sánchez, Cédula: 19.264.908
Andrea Pernil, Cédula: 32.327.658.
Pedro Diaz, Cédula: 31.887.476.
Sección: S18

Taller #1 Módulo 4 "Identificar las entidades de la BD asociadas al proyecto que escogieron para el final de la unidad curricular"

Proyecto #3 (Monitor de Integridad de Archivos - FIM)

1. MARCO TEÓRICO Y PROPÓSITO DEL MODELO DE BASE DE DATOS

Un Monitor de Integridad de Archivos (FIM) Es un mecanismo de ciberseguridad defensiva orientado a detectar modificaciones no autorizadas en archivos críticos del sistema operativo o aplicaciones de misión crítica. El objetivo central de este taller es estructurar el Modelo Entidad-Relación (DER) que sustenta el almacenamiento persistente del proyecto FIM-Secure. Este modelo permite registrar la línea base criptográfica (hashes SHA-256), gestionar directivas de escaneo, autenticar analistas y mantener una trazabilidad histórica e inalterable de todos los eventos de auditoría y alertas generadas.

2. IDENTIFICACIÓN DE ENTIDADES Y DOMINIO DE APLICACIÓN

A través del análisis funcional del sistema FIM, se han identificado cuatro (4) entidades fundamentales que componen la arquitectura de datos:

Entidad	Descripción en el Dominio FIM	Rol Operativo y Ciberseguridad
USUARIO (User)	Representa al personal analista u operador de ciberseguridad registrado en el sistema FIM.	Garantiza el control de acceso basado en roles (RBAC), controlando qué operador define directivas o consulta alertas.
POLITICA (Policy)	Agrupa reglas de auditoría y parámetros de escaneo	Permite segmentar los escaneos (ej. Servidores Web, Archivos del

	(frecuencia, nivel de severidad y catálogo de rutas).	SO, Bases de Datos) bajo un mismo perfil.
ARCHIVO (FileItem)	Almacena los metadatos y la huella digital criptográfica (SHA-256) inicial de cada fichero en monitoreo.	Actúa como la **Línea Base (Baseline)** oficial de integridad para comparar cualquier variación posterior.
ALERTA (AlertEvent)	Registra cada anomalía o discrepancia detectada durante el proceso de verificación de integridad.	Proporciona evidencias forenses sobre alteraciones, modificaciones de contenido o eliminación de ficheros.

3. JUSTIFICACIÓN TÉCNICA DE LA SELECCIÓN DE ENTIDADES

Cada entidad responde a una necesidad del estándar NIST SP 800-92 (Gestión de Registros de Seguridad). La entidad USUARIO establece la *responsabilidad y no repudio*; POLÍTICA proporciona flexibilidad operativa; ARCHIVO garantiza el principio de verificación de integridad por hashing; y ALERTA asegura la trazabilidad de incidentes* para responder a amenazas.

4. DICCIONARIO DE ATRIBUTOS, CLAVES PRIMARIAS Y FORÁNEAS

ENTIDAD: USUARIO

- **PK** id_usuario (INT, Auto-Increment)
- username (VARCHAR 50, Unique, Not Null)
- password_hash (VARCHAR 256, Not Null)
- rol (VARCHAR 20) [Admin / Analista]
- fecha_creacion (DATETIME, Default Now)

ENTIDAD: POLITICA

- **PK** id_politica (INT, Auto-Increment)
- nombre_politica (VARCHAR 100, Not Null)
- frecuencia_seg (INT, Default 3600)
- **FK** id_usuario (INT, References USUARIO

ENTIDAD: ARCHIVO (Baseline)

- **PK** id_archivo (INT, Auto-Increment)
- ruta_absoluta (VARCHAR 512, Unique, Not Null)

- sha256_base (CHAR 64, Not Null)
- tamano_bytes (BIGINT, Not Null)
- **FK** id_politica (INT, References POLITICA)

ENTIDAD: ALERTA (Incident Event)

- **PK** id_alerta (INT, Auto-Increment)
- tipo_evento (VARCHAR 20) [MODIFIED/DELETED]
- sha256_detectado (CHAR 64, Nullable)
- timestamp (DATETIME, Default Now)
- **FK** id_archivo (INT, References ARCHIVO)

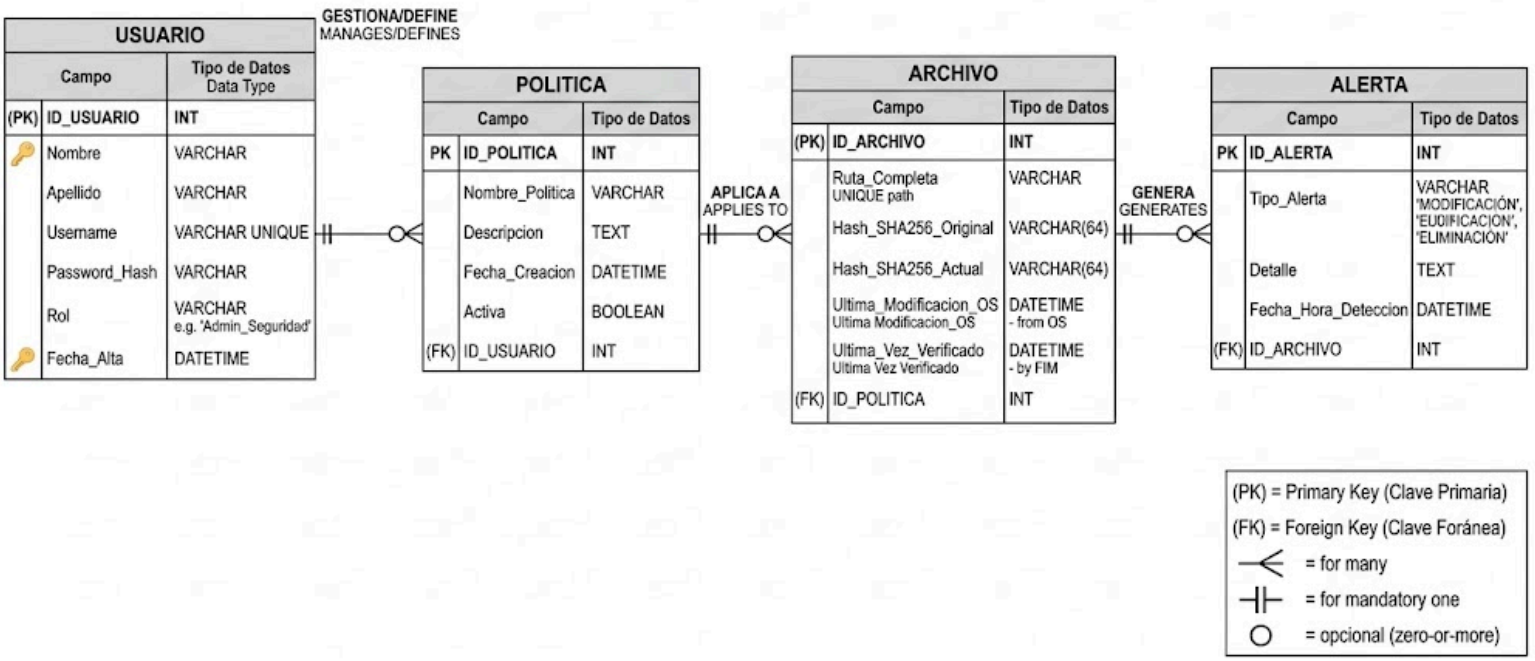
5. REGLAS DE NEGOCIO DEL SISTEMA DE BASE DE DATOS

- **RN-01 (Unicidad de Ruta):** No pueden existir dos registros de archivos con la misma ruta_absoluta dentro del sistema. La ruta actúa como identificador único en el sistema de archivos del SO.
- **RN-02 (Longitud Exacta del Hash):** Todo valor asignado a sha256_base o sha256_detectado debe ser una cadena exactamente de 64 caracteres hexadecimales.
- **RN-03 (Trazabilidad de Modificaciones):** Cuando el motor en C/Python detecta un hash diferente al almacenado en sha256_base, no sobrescribe la línea base; genera automáticamente un registro en la entidad ALERTA.
- **RN-04 (Eliminación en Cascada Controlada):** Si se elimina una política de seguridad, se eliminan en cascada los registros de archivos asociados y sus alertas históricas para mantener la integridad referencial.

6. ANÁLISIS DE CARDINALIDAD Y RELACIONES.

- **Relación USUARIO - POLITICA (1 : N):** Un `USUARIO` puede definir 1 o muchas (N) `POLITICAS` de monitoreo. Cada `POLITICA` es creada y gestionada por 1 y solo 1 `USUARIO`.
- **Relación POLITICA - ARCHIVO (1 : N):** Una `POLITICA` engloba 1 o muchos (N) `ARCHIVOS` bajo escaneo. Cada `ARCHIVO` monitoreado pertenece a 1 y solo 1 `POLITICA`.
- **Relación ARCHIVO - ALERTA (1 : N):** Un `ARCHIVO` puede generar 0, 1 o muchas (N) `ALERTAS` a lo largo del tiempo. Cada `ALERTA` corresponde a 1 y solo 1 `ARCHIVO` auditado.

7. DIAGRAMA ENTIDAD-RELACIÓN (DER / ERD)



8. CÓDIGO DE CREACIÓN DE TABLAS (DDL SQL)

SCRIPT DE CREACION DE TABLAS - FIM SECURE (PostgreSQL / SQLite)

```
CREATE TABLE USUARIO (
    id_usuario SERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(256) NOT NULL,
    rol VARCHAR(20) DEFAULT 'Analista',
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE POLITICA (
    id_politica SERIAL PRIMARY KEY,
    nombre_politica VARCHAR(100) NOT NULL,
    frecuencia_seg INT DEFAULT 3600,
```

```
    id_usuario INT NOT NULL,
     FOREIGN KEY (id_usuario) REFERENCES USUARIO(id_usuario) ON DELETE CASCADE
);

CREATE TABLE ARCHIVO (
    id_archivo SERIAL PRIMARY KEY,
    ruta_absoluta VARCHAR(512) NOT NULL UNIQUE,
    sha256_base CHAR(64) NOT NULL,
    tamano_bytes BIGINT NOT NULL,
    id_politica INT NOT NULL,
     FOREIGN KEY (id_politica) REFERENCES POLITICA(id_politica) ON DELETE CASCADE
);

CREATE TABLE ALERTA (
    id_alerta SERIAL PRIMARY KEY,
    tipo_evento VARCHAR(20) NOT NULL, -- 'MODIFIED', 'DELETED'
    sha256_detectado CHAR(64),
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    id_archivo INT NOT NULL,
     FOREIGN KEY (id_archivo) REFERENCES ARCHIVO(id_archivo) ON DELETE CASCADE
);
```